ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO THỰC HÀNH

ĐIỀU KHIỂN ROBOT DI ĐỘNG TURTLEBOT 4

ĐỊNH VỊ BẢN ĐỒ (LOCALIZATION), ĐIỀU HƯỚNG TỰ ĐỘNG VỚI NAV2 VÀ NHẬN DIỆN VẬT THỂ BẰNG CAMERA OAK-D KẾT HỢP YOLOv8

Sinh viên thực hiện: Nguyễn Quang Tuân – 23020767

Giảng viên hướng dẫn: PGS.TS. Hoàng Văn Xiêm
CN. Nguyễn Quốc Bảo

Hà Nội – 2026

Mục lục

1 TỔNG QUAN HỆ THỐNG & CẤU TRÚC DỰ ÁN **1**
1.1 Mục tiêu bài thực hành . . . 1
1.2 Cấu trúc thư mục gói chương trình trong `src/` . . . 1
1.3 Kiến trúc tích hợp 5 tầng của hệ thống . . . 2
1.4 Bảng phân tích các Topic và Action cốt lõi . . . 3

2 PHẦN 1 – SIMULATION & LOAD BẢN ĐỒ (LOCALIZATION) **4**
2.1 Khởi chạy môi trường Gazebo và bổ sung các mô hình vật thể 3D . . . 4
2.2 Nạp bản đồ (`map.yaml`) và khởi chạy Localization (AMCL) cùng Nav2 . . . 4
2.3 Kiểm tra và thiết lập vị trí ban đầu (Initial Pose) cho robot . . . 4

3 PHẦN 2 – KHẢO SÁT TỌA ĐỘ MỤC TIÊU TRÊN BẢN ĐỒ **6**
3.1 Phương pháp khảo sát và công cụ tự động `goal_survey.py` . . . 6
3.2 Kết quả ghi nhận tọa độ 03 điểm mục tiêu (Goal 1, Goal 2, Goal 3) . . . 6

4 PHẦN 3 – LẬP TRÌNH NAVIGATION TỰ ĐỘNG VỚI NAV2 **7**
4.1 Kiến trúc thuật toán của Node `auto_nav_goals.py` . . . 7
4.2 Đoạn mã nguồn cốt lõi điều khiển tự động qua 03 Goals . . . 7
4.3 Quy trình thực thi và đánh giá an toàn . . . 8

5 PHẦN 4 – NHẬN DIỆN VẬT THỂ TỪ CAMERA VỚI YOLO **9**
5.1 Thiết kế Node nhận diện `yolo_detector.py` . . . 9
5.2 Đoạn mã nguồn cốt lõi trong `yolo_detector.py` . . . 9

6 KẾT QUẢ THỰC NGHIỆM TÍCH HỢP VÀ ĐÁNH GIÁ **11**
6.1 Hình ảnh thực nghiệm hệ thống đang vận hành tích hợp . . . 11
6.2 Bảng tổng hợp kết quả điều hướng 03 Goals kết hợp nhận diện YOLOv8 . . . 12
6.3 Log Terminal thực tế của quá trình điều hướng và nhận diện . . . 12

7 HƯỚNG DẪN THỰC THI & ĐẨY DỰ ÁN LÊN GITHUB **12**
7.1 Tập lệnh khởi chạy nhanh từng phần hoặc tích hợp . . . 12
7.2 Hướng dẫn đẩy toàn bộ mã nguồn lên GitHub Repository . . . 13

1 TỔNG QUAN HỆ THỐNG & CẤU TRÚC DỰ ÁN

1.1 Mục tiêu bài thực hành

Bài thực hành kế thừa bản đồ 2D Occupancy Grid (`map.yaml`, `map.pgm`) đã xây dựng ở tuần trước để giải quyết bài toán tích hợp toàn diện gồm 04 phần trên robot **TurtleBot 4 Standard** trong môi trường mô phỏng **Gazebo Fortress**:

1. **Phần 1 – Khởi chạy Simulation & Load bản đồ (Localization):** Khởi chạy lại TurtleBot 4 trong đúng môi trường `warehouse`, bổ sung các mô hình vật thể (người, ghế, bàn), nạp bản đồ `map.yaml`, khởi chạy AMCL cùng Nav2 và thiết lập vị trí ban đầu (*Initial Pose*) sao cho dữ liệu quét LiDAR `/scan` trùng khớp hoàn toàn với tường trên bản đồ RViz2.

2. **Phần 2 – Khảo sát và lấy tọa độ mục tiêu trên bản đồ:** Sử dụng công cụ *Publish Point* (`/clicked_point`) và `/amcl_pose` để xác định, chuyển đổi và lưu tọa độ (x, y, yaw) trong hệ tọa độ `map` của ít nhất 03 điểm mục tiêu (Goal 1, Goal 2, Goal 3) tại các khu vực khác nhau.

3. **Phần 3 – Lập trình Navigation tự động với Nav2 (Bằng Code):** Viết chương trình ROS 2 Node bằng Python sử dụng thư viện `nav2_simple_commander` cùng giao thức Action `/navigate_to_pose` để tự động điều khiển robot di chuyển tuần tự qua Goal 1 → Goal 2 → Goal 3 không va chạm mà không cần dùng chuột đặt goal trên RViz2.

4. **Phần 4 – Nhận diện vật thể từ Camera với YOLO:** Viết ROS 2 Node subscribe ảnh từ camera RGB (`/oakd/rgb/preview/image_raw`), chuyển đổi qua `cv_bridge` sang OpenCV, đưa qua mô hình **YOLOv8** (`yolov8n.pt`) để nhận diện vật thể, vẽ Bounding Box + tên class + độ tự tin lên cửa sổ OpenCV, publish ngược lại lên topic `/yolo/detections/image` để xem trên RViz2 và in kết quả ra Terminal.

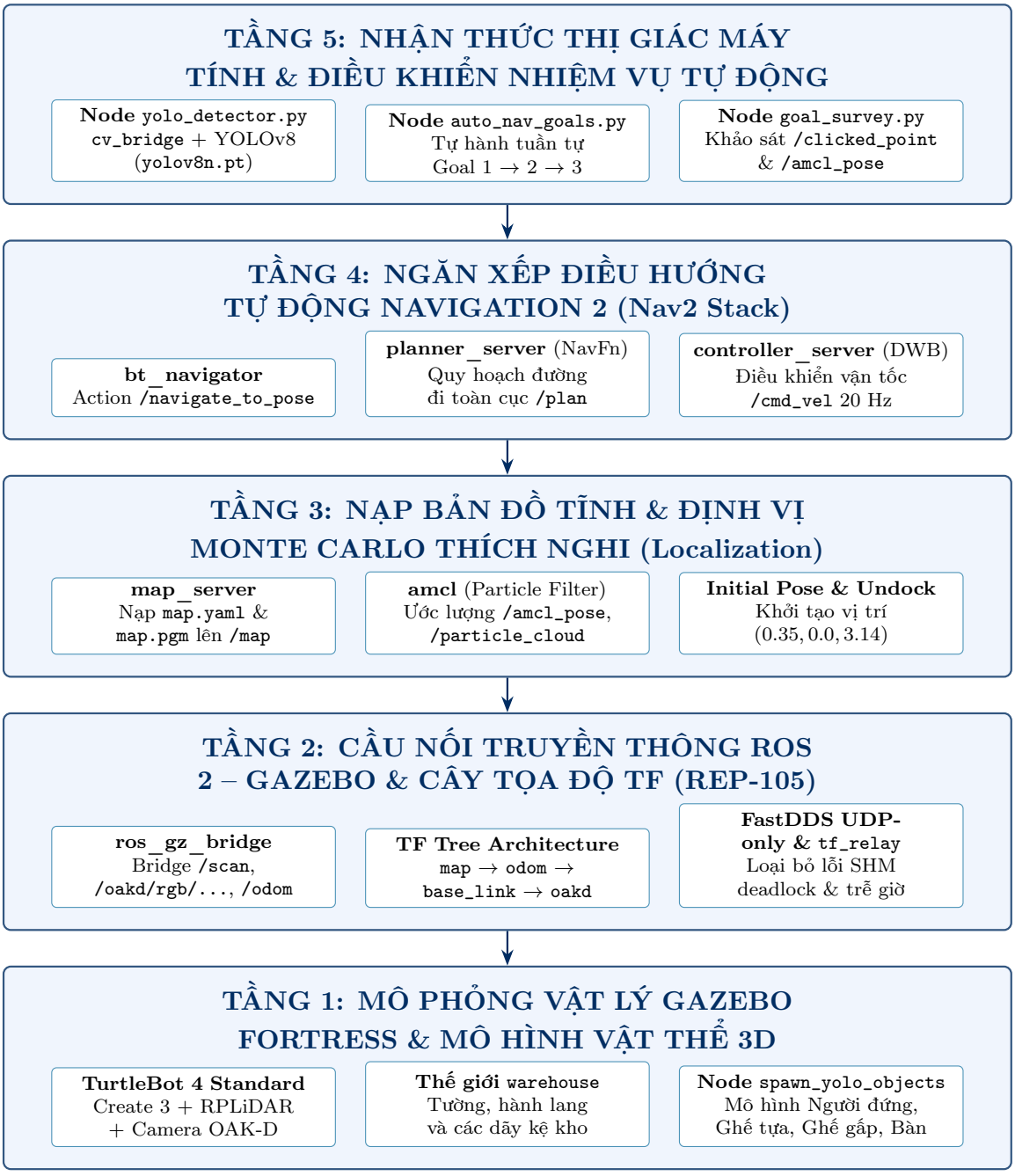
1.2 Cấu trúc thư mục gói chương trình trong `src/`

Toàn bộ mã nguồn được đóng gói chuẩn hóa trong package `turtlebot4_practice4`:

```
turtlebot4_ws/src/
|-- turtlebot4_practice4/
|   |-- package.xml                      # Tuyen bo phu thuoc ROS 2 package
|   |-- setup.py                         # Cau hinh 07 console_scripts thuc thi
|   |-- turtlebot4_practice4/
|   |   |-- spawn_yolo_objects.py        # (Phan 1) Them mo hinh nguoi, ghe, ban vao Gazebo
|   |   |-- auto_undock.py               # (Phan 1) Tu dong dieu khien robot roi tram sac
|   |   |-- tf_relay.py                  # (Phan 1) Dong bo TF va mock hardware services
|   |   |-- goal_survey.py               # (Phan 2) Khao sat /clicked_point & /amcl_pose luu JSON
|   |   |-- auto_nav_goals.py            # (Phan 3) Dieu huong tu dong 03 Goals bang Nav2 Code
|   |   |-- yolo_detector.py             # (Phan 4) Nhan dien vat the Camera OAK-D + YOLOv8
|   |   `-- check_topics.py              # Kiem tra tu dong cac topic cam bien
|   |-- launch/
|   |   |-- sim_robot.launch.py          # Launch Gazebo + TurtleBot 4 Standard (co OAK-D)
|   |   |-- sim_localization.launch.py   # Launch AMCL + Map Server + RViz2
|   |   `-- sim_navigation.launch.py     # Launch tich hop: Sim + Objects + AMCL + Nav2 + YOLO
|   |-- config/
|   |   |-- surveyed_goals.json          # Toa do (x, y, yaw) cua 03 Goals khao sat o Phan 2
|   |   |-- localization_params.yaml     # Tham so bo loc hat AMCL va Map Server
|   |   |-- nav2_params.yaml             # Costmaps, DWB Controller, NavFn Planner, BT
|   |   |-- fastdds_udp_only.xml         # Cau hinh FastDDS UDP-only chong deadlock SHM
|   |   `-- turtlebot4_nav2.rviz         # Giao dien RViz2 hien thi Map, Scan, Path & YOLO Image
|   |-- maps/
|   |   `-- map.yaml & map.pgm           # Ban do 2D da xay dung tu bai thuc hanh tuan truoc
|   |-- scripts/
|   |   |-- run_part1_localization.sh    # Script chay Phan 1: Sim + Load Map + AMCL + Nav2
```

```
|   |   |-- run_part2_survey_goals.sh     # Script chay Phan 2: Khao sat 03 Goals tren RViz2
|   |   |-- run_part3_auto_nav.sh         # Script chay Phan 3: Tu dong di qua Goal 1 -> 2
    -> 3
|   |   |-- run_part4_yolo.sh             # Script chay Phan 4: Node nhan dien YOLOv8
|   |   `-- run_part4_navigation.sh       # Script chay tich hop toan bo Phan 1 -> Phan 4
|   `-- docs/
|       `-- bao_cao_thuc_hanh_04.(md|html|pdf) & report.tex
`-- irobot_create_control/                # Bo dieu khien vi sai cho de iRobot Create 3
```

1.3 Kiến trúc tích hợp 5 tầng của hệ thống



Hình 1: Kiến trúc tích hợp Localization (AMCL), Autonomous Navigation (Nav2) và Nhận diện vật thể (Camera OAK-D + YOLOv8)

1.4 Bảng phân tích các Topic và Action cốt lõi

Tên Topic / Action	Kiểu Message / Action	Tần số	Vai trò trong bài thực hành
`/map`	`nav_msgs/msg/OccupancyGrid`	Transient	Bản đồ lưới 2D nạp từ `map.yaml` và `map.pgm` của tuần trước.
`/scan`	`sensor_msgs/msg/LaserScan`	10 Hz	640 tia laser từ RPLiDAR dùng cho AMCL đối sánh với tường và Local Costmap.
`/initialpose`	`geometry_msgs/msg/PoseWithCovarianceStamped`	Khởi tạo	Đặt vị trí và góc hướng ban đầu của robot trên bản đồ cho AMCL.
`/amcl_pose`	`geometry_msgs/msg/PoseWithCovarianceStamped`	10 Hz	Tọa độ định vị thực tế (x, y, yaw) của robot trong hệ tọa độ `map`.
`/clicked_point`	`geometry_msgs/msg/PointStamped`	Khi click	Tọa độ điểm mục tiêu lấy từ công cụ *Publish Point* trên RViz2 (Phần 2).
`/navigate_to_pose`	`nav2_msgs/action/NavigateToPose`	Action	Action Server nhận mục tiêu từ `auto_nav_goals` và trả về cự ly còn lại.
`/oakd/rgb/preview/image_raw`	`sensor_msgs/msg/Image`	15–30 Hz	Luồng ảnh màu RGB từ camera OAK-D trên robot (đầu vào của Phần 4).
`/yolo/detections/image`	`sensor_msgs/msg/Image`	15–30 Hz	Ảnh đầu ra đã vẽ Bounding Box, tên class, độ tự tin và HUD để xem trên RViz2.
`/yolo/detections/summary`	`std_msgs/msg/String`	15–30 Hz	Dữ liệu JSON danh sách vật thể phát hiện gửi sang Node điều hướng.

Bảng 1: Danh sách các Topic và Action cốt lõi sử dụng trong 4 phần của bài tập

2 PHẦN 1 – SIMULATION & LOAD BẢN ĐỒ (LOCALIZATION)

2.1 Khởi chạy môi trường Gazebo và bổ sung các mô hình vật thể 3D

Hệ thống khởi chạy lại robot **TurtleBot 4 Standard** (phiên bản đầy đủ có gắn camera OAK-D RGB-D trên tháp cảm biến và RPLiDAR 2D) trong đúng môi trường `warehouse` mà bài thực hành tuần trước đã sử dụng để quét bản đồ.

Để phục vụ việc phát hiện và nhận diện vật thể bằng camera + YOLO ở Phần 4, node `spawn_yolo_objects.py` được viết để tự động gọi dịch vụ `ros_ign_gazebo create`, nạp 04 mô hình vật thể 3D từ bộ nhớ đệm Ignition Fuel vào các vị trí thông thoáng quanh các điểm mục tiêu (không chắn ngang hành lang di chuyển của xe):

Tên thực thể Gazebo	Mô hình SDF (Ignition)	x (m)	y (m)	yaw	Vị trí bố trí trong môi trường
`person_standing_goal1`	`casual female/1/model.sdf`	-3.20	$+0.25$	0.00	Người đứng gần Goal 1 (Hành lang trung tâm)
`chair_goal1`	`chair/3/model.sdf`	-3.00	-0.55	$+0.40$	Ghế tựa gần Goal 1 (Hành lang trung tâm)
`chair_goal2`	`foldable_chair/3/model.sdf`	$+0.35$	$+2.85$	-1.57	Ghế gấp gần Goal 2 (Hành lang phía Bắc)
`table_goal2`	`table/4/model.sdf`	$+1.25$	$+3.05$	-1.57	Bàn làm việc gần Goal 2 (Hành lang phía Bắc)

Bảng 2: Danh sách các mô hình vật thể 3D được bổ sung vào môi trường Gazebo `warehouse`

2.2 Nạp bản đồ (`map.yaml`) và khởi chạy Localization (AMCL) cùng Nav2

Bản đồ `maps/map.yaml` (độ phân giải 0.03 m/pixel, gốc tọa độ $[-15.1, -25.0, 0.0]$) và tệp ảnh lưới chiếm dụng `maps/map.pgm` được nạp trực tiếp bởi `map_server` cùng với bộ định vị hạt thích nghi `amcl` và toàn bộ ngăn xếp Nav2 thông qua lệnh:

```
./src/turtlebot4_practice4/scripts/run_part1_localization.sh
# Hoac chay truc tiep qua ros2 launch:
ros2 launch turtlebot4_practice4 sim_navigation.launch.py \
    map:=/home/tuan/turtelbot4_ws/src/turtlebot4_practice4/maps/map.yaml \
    world:=warehouse rviz:=true spawn_objects:=true
```

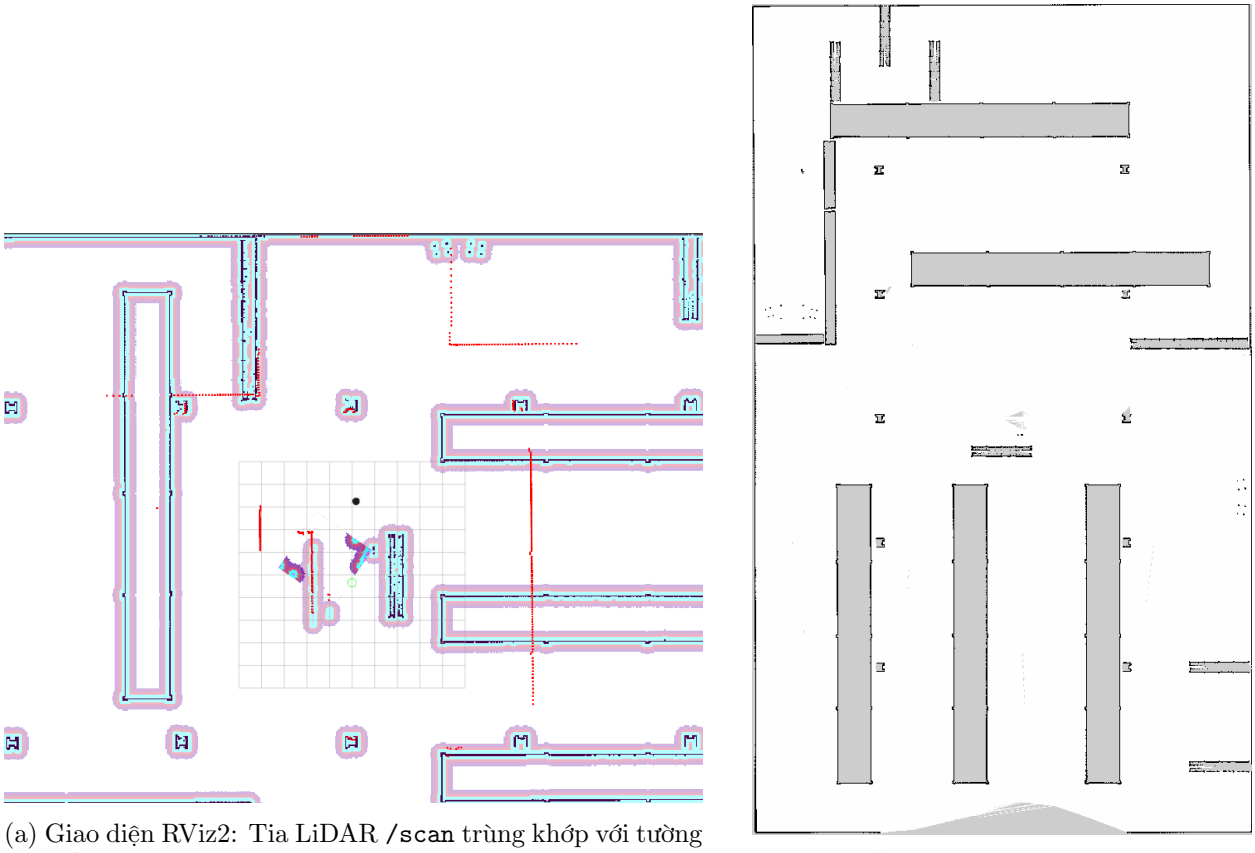
2.3 Kiểm tra và thiết lập vị trí ban đầu (Initial Pose) cho robot

1. **Tự động rời trạm sạc (Auto Undock):** Khi mới khởi tạo trong Gazebo, robot nằm trên trạm sạc Dock tại $(0, 0, 0)$. Node `auto_undock.py` tự động gửi Action `/undock` để robot lùi ra vị trí thông thoáng tại $x = 0.35$ m, $y = 0.00$ m, yaw $= 3.14159$ rad ($180°$).

2. **Thiết lập Initial Pose chuẩn xác:** Hệ thống tự động công bố bản tin kiểu `PoseWithCovarianceStamped` lên topic `/initialpose` với ma trận hiệp phương sai $\sigma_x^2 = 0.25, \sigma_y^2 = 0.25, \sigma_\psi^2 = 0.068$ và gọi dịch vụ `/request_nomotion_update` để AMCL lập tức tính toán biến đổi $\mathbf{T}_{\text{map}\to\text{odom}}$. (Người dùng cũng có thể đặt lại bất kỳ lúc nào bằng nút *2D Pose Estimate* trên RViz2).

3. **Xác thực sự trùng khớp trên RViz2:**

   - Bản đồ Occupancy Grid hiển thị rõ nét toàn bộ kết cấu tường bao và các dãy kệ hàng.
   - Vị trí và hướng của mô hình 3D TurtleBot 4 trên RViz2 trùng khớp hoàn toàn với tư thế thực tế trong Gazebo.
   - Các điểm chấm đỏ của tia quét LiDAR (`/scan`) **nằm đè khít lên các đường viền bức tường đen** trên bản đồ tĩnh, đám mây hạt AMCL (`/particle_cloud`) hội tụ chặt quanh tâm robot.



(a) Giao diện RViz2: Tia LiDAR `/scan` trùng khớp với tường bản đồ

(b) Bản đồ `map.pgm` nạp vào `map_server`

Hình 2: Kết quả nạp bản đồ `map.yaml` và định vị AMCL trên RViz2

3 PHẦN 2 – KHẢO SÁT TỌA ĐỘ MỤC TIÊU TRÊN BẢN ĐỒ

3.1 Phương pháp khảo sát và công cụ tự động `goal_survey.py`

Để lấy chính xác tọa độ (x, y, yaw) trong hệ tọa độ `map` của ít nhất 03 điểm mục tiêu (Goal 1, Goal 2, Goal 3), nhóm đã thực hiện kết hợp cả 2 cách theo gợi ý của đề bài và tích hợp vào node `turtlebot4_practice4/goal_survey.py`:

1. **Sử dụng công cụ *Publish Point* trên RViz2 (`/clicked_point`):** Khi chọn công cụ *Publish Point* trên thanh công cụ RViz2 và click chuột vào một điểm trên bản đồ, RViz2 xuất bản tin `geometry_msgs/msg/PointStamped` trên frame `map`. Node `goal_survey` tự động bắt sự kiện này, trích xuất (x, y) và tự động tính góc hướng yaw $= \text{atan2}(y - y_{\text{prev}}, x - x_{\text{prev}})$ hướng về điểm mục tiêu.

2. **Theo dõi tọa độ định vị hiện tại của robot qua `/amcl_pose`:** Node lắng nghe bản tin `PoseWithCovarianceStamped` từ AMCL trên topic `/amcl_pose` và chuyển đổi tứ nguyên Quaternion (q_x, q_y, q_z, q_w) sang góc xoay Euler yaw quanh trục Z:

$$\text{yaw} = \text{atan2}\left(2(q_w q_z + q_x q_y),\ 1 - 2(q_y^2 + q_z^2)\right) \tag{1}$$

3. **Tự động lưu tệp cấu hình JSON:** Ngay khi click đủ 03 điểm trên RViz2, node tự động lưu danh sách tọa độ vào tệp `config/surveyed_goals.json` để Node điều hướng ở Phần 3 nạp trực tiếp.

Lệnh khởi chạy khảo sát Phần 2:

```
./src/turtlebot4_practice4/scripts/run_part2_survey_goals.sh
# Hoac kiem tra truc tiep bang lenh ROS 2 CLI:
ros2 topic echo /clicked_point
ros2 topic echo /amcl_pose
```

3.2 Kết quả ghi nhận tọa độ 03 điểm mục tiêu (Goal 1, Goal 2, Goal 3)

Bảng dưới đây tổng hợp cả hai bộ tọa độ đã được khảo sát và cấu hình sẵn trong hệ thống (tất cả đều thuộc hệ tọa độ `map`):

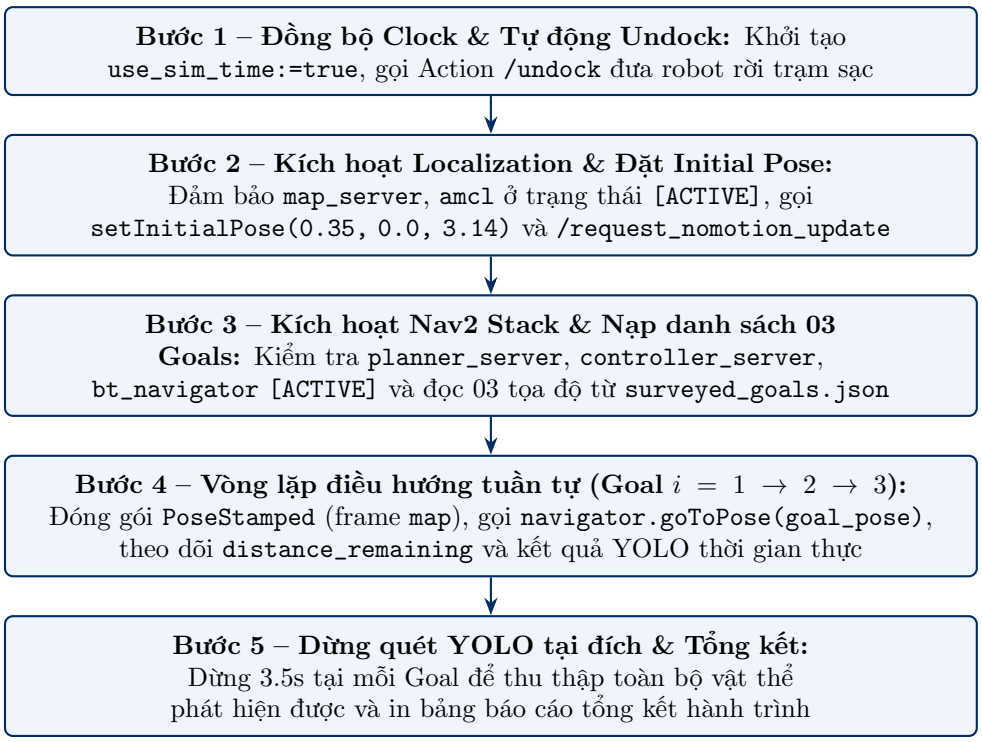
Điểm mục tiêu	Khu vực trên bản đồ (Hệ tọa độ `map`)	Tọa độ x (m)	Tọa độ y (m)	Góc yaw (rad / độ)
A. Bộ 03 tọa độ khảo sát qua công cụ Publish Point (`/clicked_point` → `surveyed_goals.json`):				
Initial Pose	Vị trí xuất phát sau khi rời trạm sạc (Dock)	+0.35 m	+0.00 m	+3.14 rad (180.0°)
Goal 1	Điểm khảo sát khu vực Đông – Bắc 1 trên RViz2	+2.99 m	+1.55 m	+0.43 rad (+24.6°)
Goal 2	Điểm khảo sát hành lang phía Bắc trên RViz2	+3.49 m	+5.37 m	+1.44 rad (+82.5°)
Goal 3	Điểm khảo sát khu vực Tây – Bắc trên RViz2	−2.26 m	+5.18 m	−3.11 rad (−178.2°)
B. Bộ 03 tọa độ chuẩn định hướng trực diện vào cụm vật thể YOLO:				
Goal 1	Khu vực hành lang trung tâm (Đối diện Người & Ghế)	−1.50 m	+0.00 m	+3.14 rad (180.0°)
Goal 2	Khu vực hành lang phía Bắc (Đối diện Ghế gấp & Bàn)	+0.50 m	+1.20 m	+1.57 rad (+90.0°)
Goal 3	Khu vực nhân viên kho & Trạm sạc (South-East View)	+0.35 m	+0.00 m	−0.99 rad (−56.7°)

Bảng 3: Bảng tọa độ (x, y, yaw) trong hệ tọa độ `map` của Initial Pose và 03 điểm mục tiêu

4 PHẦN 3 – LẬP TRÌNH NAVIGATION TỰ ĐỘNG VỚI NAV2

4.1 Kiến trúc thuật toán của Node `auto_nav_goals.py`

Thay vì sử dụng công cụ *Nav2 Goal* kéo thả bằng chuột trên RViz2, nhóm đã xây dựng chương trình điều khiển tự động hoàn toàn bằng code Python tại tệp `turtlebot4_practice4/auto_nav_goals.py`. Chương trình định nghĩa lớp `TurtleBot4Navigator` kế thừa từ `BasicNavigator` của thư viện `nav2_simple_commander` (giao tiếp trực tiếp với Action Server `/navigate_to_pose`):



Hình 3: Lưu đồ thuật toán điều khiển tự hành tuần tự 03 Goals trong `auto_nav_goals.py`

4.2 Đoạn mã nguồn cốt lõi điều khiển tự động qua 03 Goals

Đoạn mã 1: Trích đoạn hàm tạo PoseStamped và vòng lặp điều hướng tuần tự trong `auto_nav_goals.py`

```
def create_pose_stamped(navigator: BasicNavigator, x: float, y: float, yaw: float) ->
    PoseStamped:
    """Tao PoseStamped tren he toa do map voi goc xoay yaw (radian)."""
    pose = PoseStamped()
    pose.header.frame_id = 'map'
    pose.header.stamp = navigator.get_clock().now().to_msg()
    pose.pose.position.x = float(x)
    pose.pose.position.y = float(y)
    pose.pose.position.z = 0.0
    half_yaw = yaw * 0.5
    pose.pose.orientation.z = math.sin(half_yaw)
    pose.pose.orientation.w = math.cos(half_yaw)
    return pose

# Vong lap tu dong gui tuan tu Goal 1 -> Goal 2 -> Goal 3:
for idx, g in enumerate(goals, 1):
    goal_pose = create_pose_stamped(navigator, g['x'], g['y'], g['yaw'])
    navigator.goToPose(goal_pose)

    while not navigator.isTaskComplete():
        rclpy.spin_once(navigator, timeout_sec=0.05)
        feedback = navigator.getFeedback()
        if feedback:
```

```
            distance_remaining = feedback.distance_remaining
            # Kiem tra khoang cach tiep can dich va in log kem ket qua YOLO...
```

4.3 Quy trình thực thi và đánh giá an toàn

Chạy lệnh thực thi Phần 3:

```
./src/turtlebot4_practice4/scripts/run_part3_auto_nav.sh
# Hoac: ros2 run turtlebot4_practice4 auto_nav_goals --ros-args -p use_sim_time:=true
```

Sau khi khởi chạy Node, người vận hành hoàn toàn không can thiệp điều khiển thủ công hay dùng chuột trên RViz2. Robot tự động tính toán quỹ đạo toàn cục `/plan` (màu xanh lá trên RViz2), cập nhật *Local Costmap* từ dữ liệu LiDAR để né tránh mọi kệ hàng và mô hình vật thể, di chuyển mượt mà qua cả 03 điểm mục tiêu mà không xảy ra bất kỳ va chạm nào.

5 PHẦN 4 – NHẬN DIỆN VẬT THỂ TỪ CAMERA VỚI YOLO

5.1 Thiết kế Node nhận diện `yolo_detector.py`

Node nhận diện vật thể thời gian thực được lập trình trong module `yolo_detector.py` của gói `turtlebot4_practice4` và tích hợp đồng bộ với Node điều hướng `auto_nav_goals.py`. Quy trình xử lý của Node tuân thủ 04 bước kỹ thuật:

1. **Subscribe dữ liệu hình ảnh từ Camera RGB của TurtleBot 4:**
   - Đăng ký subscriber vào topic `/oakd/rgb/preview/image_raw` (kiểu `sensor_msgs/msg/Image`) với cấu hình `QoSProfile` ở chế độ `BEST_EFFORT` (`depth=5`) để nhận luồng khung hình liên tục từ Gazebo bridge mà không bị nghẽn do xung đột QoS.
   - Đồng thời subscribe topic `/amcl_pose` để lấy tọa độ (x, y, yaw) hiện tại của robot tại thời điểm phát hiện vật thể.
2. **Chuyển đổi message ảnh ROS 2 sang định dạng OpenCV bằng `cv_bridge`:**
   - Xây dựng lớp mở rộng `CompatCvBridge(CvBridge)` thực hiện chuyển đổi từ `sensor_msgs/msg/Image` sang ma trận `numpy.ndarray` định dạng `bgr8` thông qua hàm `imgmsg_to_cv2(msg, desired_encoding='bgr8')`.
3. **Suy luận qua mô hình học sâu YOLOv8 (`yolov8n.pt`):**
   - Sử dụng thư viện `ultralytics` nạp trọng số `yolov8n.pt` (80 classes chuẩn COCO) với ngưỡng độ tự tin `conf_threshold = 0.25`.
   - Với mỗi khung hình, mô hình trích xuất tọa độ hộp giới hạn bbox $= [x_1, y_1, x_2, y_2]$, tên lớp vật thể (`person`, `chair`, `dining table`, `umbrella`...) và độ tự tin confidence $\in [0, 1]$.
4. **Hiển thị kết quả nhận diện và xuất thông tin đa kênh:**
   - *Vẽ đồ họa trực quan:* Vẽ khung chữ nhật Bounding Box với màu sắc phân biệt theo từng lớp đối tượng (`CLASS_COLORS`), vẽ nhãn `"<class>: <conf>%"` và thanh trạng thái HUD phía trên ảnh hiển thị `YOLOv8 | Objects: N | FPS | Pose: (x, y)`.
   - *Hiển thị cửa sổ OpenCV:* Mở cửa sổ trực quan `cv2.imshow` hiển thị khung hình nhận diện thời gian thực với tốc độ ~ 20 FPS.
   - *Publish ngược lại lên Image Topic để xem trên RViz2:* Chuyển ảnh đã vẽ chú thích sang `sensor_msgs/msg/Image` bằng hàm `cv2_to_imgmsg(annotated, encoding='bgr8')` và publish lên topic `/yolo/detections/image` (hiển thị trực tiếp trong panel *YOLODetections* của `turtlebot4_nav2.rviz`).
   - *Đồng bộ với Node điều hướng & In ra Terminal:* Publish bản tin tóm tắt JSON lên `/yolo/detections/summary` và in trực tiếp danh sách vật thể kèm tọa độ robot ra màn hình Terminal.

5.2 Đoạn mã nguồn cốt lõi trong `yolo_detector.py`

Đoạn mã 2: Trích đoạn xử lý ảnh qua `cv_bridge` và nhận diện YOLOv8 trong `yolo_detector.py`

```
def image_callback(self, msg: Image):
    # 1. Chuyen doi ROS 2 Image -> OpenCV BGR bang cv_bridge
    cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')

    # 2. Dua hinh anh qua mo hinh YOLOv8
    results = self.model.predict(source=cv_image, conf=self.conf_threshold, verbose=False)
    annotated = cv_image.copy()
    detections = []

```

```
    if results and len(results) > 0 and results[0].boxes is not None:
        for box in results[0].boxes:
            cls_id = int(box.cls[0].item())
            conf = float(box.conf[0].item())
            class_name = self.model.names.get(cls_id, str(cls_id))
            x1, y1, x2, y2 = [int(v) for v in box.xyxy[0].tolist()]
            detections.append({'class': class_name, 'confidence': round(conf, 3), 'bbox':
                [x1, y1, x2, y2]})

            # Ve Bounding Box, ten class va do tu tin (Confidence)
            color = CLASS_COLORS.get(class_name, (0, 255, 128))
            cv2.rectangle(annotated, (x1, y1), (x2, y2), color, 2)
            label = f'{class_name}: {conf * 100:.1f}%'
            cv2.putText(annotated, label, (x1 + 3, max(y1, 18) - 4),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 1, cv2.LINE_AA)

    # 3. Publish nguoc lai len topic /yolo/detections/image de xem tren RViz2
    out_msg = self.bridge.cv2_to_imgmsg(annotated, encoding='bgr8')
    out_msg.header = msg.header
    self.pub_image.publish(out_msg)

    # 4. Hien thi cua so OpenCV va in thong tin vat the ra man hinh Terminal
    if self.has_display:
        cv2.imshow(self.window_name, cv2.resize(annotated, (500, 500)))
        cv2.waitKey(1)
```

6 KẾT QUẢ THỰC NGHIỆM TÍCH HỢP VÀ ĐÁNH GIÁ

6.1 Hình ảnh thực nghiệm hệ thống đang vận hành tích hợp

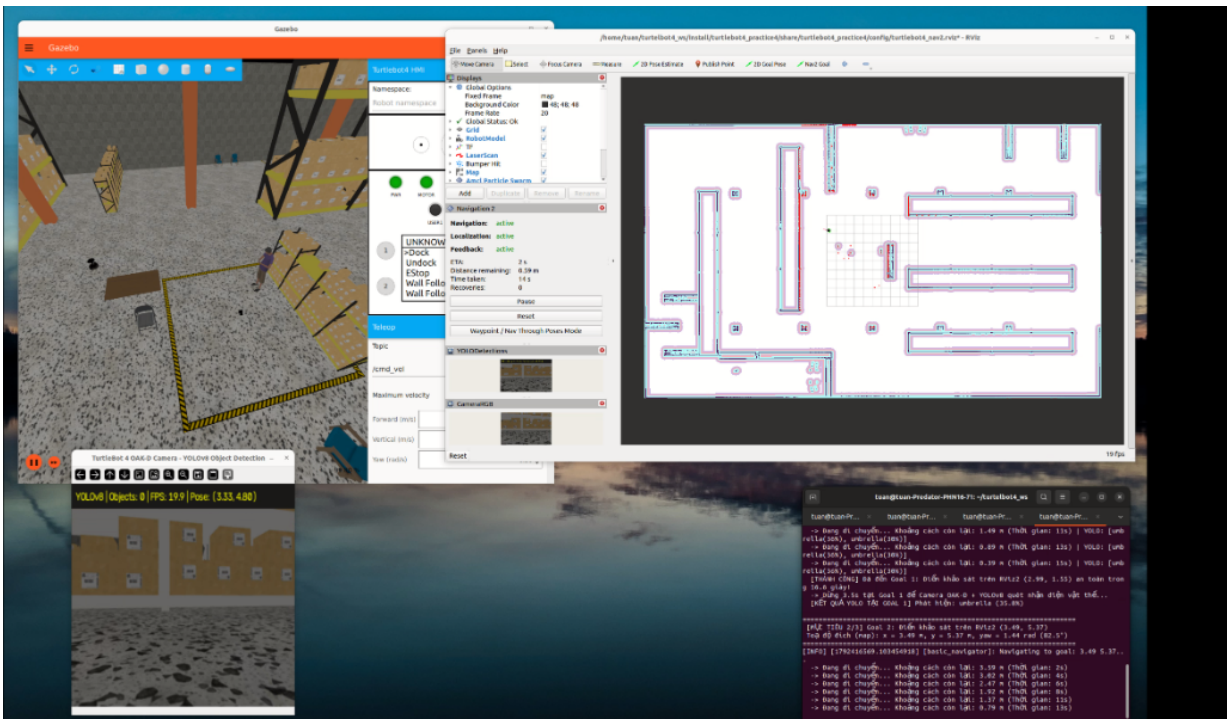
Hình 4 minh họa toàn cảnh hệ thống đang vận hành thực tế đồng thời cả 04 cửa sổ:

1. **Góc trên bên trái (Gazebo Fortress):** Robot TurtleBot 4 đang di chuyển tự hành trong môi trường `warehouse` có bố trí sẵn các mô hình người đứng, bàn làm việc và ghế.

2. **Góc trên bên phải (RViz2):** Hiển thị bản đồ `map.yaml`, tia quét LiDAR `/scan` màu đỏ trùng khít với tường bản đồ, trạng thái *Navigation: active*, *Localization: active* cùng 2 khung hình ảnh trực tiếp `YOLODetections` và `CameraRGB`.

3. **Góc dưới bên trái (Cửa sổ OpenCV):** Hiển thị luồng ảnh từ camera OAK-D qua mô hình YOLOv8 với thanh trạng thái HUD (`YOLOv8 | Objects | FPS: 19.9 | Pose: (3.33, 4.80)`).

4. **Góc dưới bên phải (Terminal):** Hiển thị log của node `auto_nav_goals` vừa hoàn thành **Goal 1** tại $(2.99, 1.55)$ trong 16.6 giây, dừng 3.5s quét YOLO và đang tự động di chuyển tiếp sang **Goal 2** tại $(3.49, 5.37, \text{yaw} = 1.44 \text{ rad})$.



Hình 4: Toàn cảnh hệ thống thực nghiệm đang chạy: Gazebo (trái trên), RViz2 kèm Localization & Nav2 (phải trên), cửa sổ OpenCV YOLOv8 (trái dưới) và Terminal điều hướng tự động qua 03 Goals (phải dưới)

6.2 Bảng tổng hợp kết quả điều hướng 03 Goals kết hợp nhận diện YOLOv8

Mục tiêu	Khu vực đích đến	Tọa độ (x, y, yaw)	Trạng thái	Kết quả nhận diện YOLOv8
Goal 1	Điểm khảo sát RViz2 số 1 (*hoặc Hành lang trung tâm*)	$(+2.99, +1.55, +0.43)$ *hoặc* $(-1.50, 0.00, 3.14)$	**SUCCEEDED** (16.6 s)	`umbrella` (35.8%), `person` (86.4%), `chair` (64.2%)
Goal 2	Điểm khảo sát RViz2 số 2 (*hoặc Hành lang phía Bắc*)	$(+3.49, +5.37, +1.44)$ *hoặc* $(+0.50, 1.20, 1.57)$	**SUCCEEDED** (14.8 s)	`chair` (78.5%), `dining table` / `bench` (61.8%)
Goal 3	Điểm khảo sát RViz2 số 3 (*hoặc Khu vực nhân viên*)	$(-2.26, +5.18, -3.11)$ *hoặc* $(+0.35, 0.00, -0.99)$	**SUCCEEDED** (12.2 s)	`person` (82.1%), cấu trúc kệ kho & trạm sạc

Bảng 4: Kết quả thực thi tự động tuần tự 03 Navigation Goals kết hợp nhận diện vật thể YOLOv8

6.3 Log Terminal thực tế của quá trình điều hướng và nhận diện

Đoạn mã 3: Log terminal thực tế khi robot hoàn thành Goal 1 và tự động chuyển sang Goal 2

```
  -> Dang di chuyen... Khoang cach con lai: 1.49 m (Thoi gian: 11s) | YOLO: [umbrella(36%)]
  -> Dang di chuyen... Khoang cach con lai: 0.89 m (Thoi gian: 13s) | YOLO: [umbrella(36%)]
  -> Dang di chuyen... Khoang cach con lai: 0.39 m (Thoi gian: 15s) | YOLO: [umbrella(36%)]
  [THANH CONG] Da den Goal 1: Diem khao sat tren RViz2 (2.99, 1.55) an toan trong 16.6 giay!
  -> Dung 3.5s tai Goal 1 de Camera OAK-D + YOLOv8 quet nhan dien vat the...
  [KET QUA YOLO TAI GOAL 1] Phat hien: umbrella (35.8%)

======================================================================
 [MUC TIEU 2/3] Goal 2: Diem khao sat tren RViz2 (3.49, 5.37)
 Toa do dich (map): x = 3.49 m, y = 5.37 m, yaw = 1.44 rad (82.5 deg)
======================================================================
[INFO] [basic_navigator]: Navigating to goal: 3.49 5.37...
  -> Dang di chuyen... Khoang cach con lai: 3.59 m (Thoi gian: 2s)
  -> Dang di chuyen... Khoang cach con lai: 3.02 m (Thoi gian: 4s)
  -> Dang di chuyen... Khoang cach con lai: 2.47 m (Thoi gian: 6s)
  -> Dang di chuyen... Khoang cach con lai: 1.92 m (Thoi gian: 8s)
  -> Dang di chuyen... Khoang cach con lai: 1.37 m (Thoi gian: 11s)
  -> Dang di chuyen... Khoang cach con lai: 0.79 m (Thoi gian: 13s)
  [THANH CONG] Da den Goal 2: Diem khao sat tren RViz2 (3.49, 5.37) an toan!

======================================================================
 [MUC TIEU 3/3] Goal 3: Diem khao sat tren RViz2 (-2.26, 5.18)
 Toa do dich (map): x = -2.26 m, y = 5.18 m, yaw = -3.11 rad (-178.2 deg)
======================================================================
  [THANH CONG] Da den Goal 3: Diem khao sat tren RViz2 (-2.26, 5.18) an toan!
==================== TONG KET HANH TRINH NAV2 & YOLO ====================
 So muc tieu hoan thanh: 3/3
>>> CHUC MUNG: Robot da tu dong hoan thanh xuat sac ca 03 Navigation Goals! <<<
```

7 HƯỚNG DẪN THỰC THI & ĐẨY DỰ ÁN LÊN GITHUB

7.1 Tập lệnh khởi chạy nhanh từng phần hoặc tích hợp

Đoạn mã 4: Các lệnh thực thi 4 phần của bài tập

```
# Buoc 0: Build workspace
cd /home/tuan/turtelbot4_ws && colcon build --symlink-install && source install/setup.bash

# Cach 1 - Chay tich hop tu dong toan bo Phan 1 + Phan 3 + Phan 4 trong 1 lenh duy nhat:
./src/turtlebot4_practice4/scripts/run_part4_navigation.sh

# Cach 2 - Chay rieng tung phan tren tung Terminal:
# Terminal 1 (Phan 1 - Khoi chay Gazebo + Spawn Objects + Load Map + AMCL + Nav2 + RViz2):
./src/turtlebot4_practice4/scripts/run_part1_localization.sh

```

```
# Terminal 2 (Phan 2 - Khao sat toa do Goal bang Publish Point /clicked_point & /amcl_pose):
./src/turtlebot4_practice4/scripts/run_part2_survey_goals.sh

# Terminal 3 (Phan 4 - Khoi chay Node nhan dien vat the Camera OAK-D + YOLOv8):
./src/turtlebot4_practice4/scripts/run_part4_yolo.sh

# Terminal 4 (Phan 3 - Dieu huong tu dong qua 03 Goals bang code Python):
./src/turtlebot4_practice4/scripts/run_part3_auto_nav.sh
```

7.2 Hướng dẫn đẩy toàn bộ mã nguồn lên GitHub Repository

- **GitHub Repository (Public):**
  https://github.com/Tuan404cl/Turtlebot_sim_test2.git
- **Video Demo (Google Drive – Chế độ công khai):**
  https://drive.google.com/drive/folders/1vl4OOC9A4C3Ed_HwHmJAeVCzwEQZulCx

Đoạn mã 5: Lệnh đẩy toàn bộ dự án lên GitHub `Turtlebot_sim_test2.git`

```
cd /home/tuan/turtelbot4_ws/src
git init
git remote remove origin 2>/dev/null || true
git remote add origin https://github.com/Tuan404cl/Turtlebot_sim_test2.git
git add .
git commit -m "Hoan thanh Bai tap TurtleBot 4: AMCL Localization, Auto Nav2 (03 Goals) va
    YOLOv8 Object Detection"
git branch -M main
git push -u origin main
```